

Statistic Vocabularies

Silvia Tommaso

RomaTre Univeristy

1 Introduction

Eurostat statistical tables provide quantitative data about a wide range of social, economic, environmental, and demographic phenomena.

This assignment focuses on constructing and organizing a vocabulary from the textual information contained in a collection of 7,604 such tables, classifying the extracted terms into four categories, **Measures**, **Dimension Names**, **Dimension Values** and **Unit Information**, and grouping the measures into domains according to their semantic similarity.

2 Methodology

The work is organized into three main steps. The first two cover the construction and classification of vocabulary V and are implemented together through a single nested procedure based on heuristics. Finally, the measures are grouped into clusters based on their hierarchical relationships and assigned to one of eight domains from a manually curated list, with the aid of an LLM accessed via API.

The implementation of the following procedures is provided in Appendix A.

2.1 Vocabulary construction and classification

	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P	Q	R
1	Time frequency	Unit of measure	Type of vessel	Transport coverage	Geopolitical entity (reporting)	Time												
2	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Austria	2012.0	2013.0	2014.0	2015.0	2016.0	2017.0	2018.0	2019.0	2020.0	2021.0	2022.0	2023.0	
3	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Belgium	2191.0	2353.0	2177.0	1806.0	1962.0	2022.0	1489.0	1715.0	1606.0	1506.0	1233.0	1187.0	
4	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Bulgaria	1943.0	10365.0	10451.0	10426.0	10331.0	11099.0	7989.0	7787.0	7386.0	8130.0	7559.0	6952.0	
5	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Czechia	5349.0	5374.0	5074.0	5595.0	5477.0	5279.0	4658.0	5601.0	6256.0	5782.0	3787.0	4253.0	
6	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Germany	38.0	25.0	27.0	33.0	36.0	25.0	23.0	32.0	18.0	22.0	20.0	18.0	
7	Annual	Million tonne-kilometre (TKM)	Total	Total transport	European Union - 27 countries (from 2020)	58488.0	60707.0	58993.0	53335.0	54347.0	59318.0	66921.0	52953.0	46238.0	46397.0	44117.0	41506.0	
8	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Finland	148832.0	151305.0	149431.0	145505.0	145287.0	145986.0	130051.0	138908.0	130478.0	135047.0	121971.0	116495.0	
9	Annual	Million tonne-kilometre (TKM)	Total	Total transport	France													
10	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Croatia	8981.0	9082.0	8770.0	8484.0	8310.0	7515.0	7255.0	7996.0	6988.0	7265.0	6593.0	5920.0	
11	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Hungary	772.0	771.0	718.0	879.0	836.0	813.0	678.0	835.0	903.0	841.0	595.0	486.0	
12	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Luxembourg	1462.0	1824.0	1811.0	1824.0	1875.0	1962.0	1608.0	2120.0	1989.0	1873.0	1508.0	1316.0	
13	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Netherlands	290.0	313.0	285.0	235.0	190.0	195.0	205.0	228.0	201.0	217.0	200.0	185.0	
14	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Poland	46681.0	47688.0	48252.0	48922.0	47987.0	47960.0	45886.0	46184.0	44206.0	46991.0	44951.0	41611.0	
15	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Romania	131.0	91.0	110.0	88.0	108.0	115.0	125.0	87.0	77.0	54.0	40.0	29.0	
16	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Sweden	12520.0	12242.0	11790.0	13388.0	13153.0	12517.0	12261.0	11997.0	13638.0	13522.0	10797.0	11947.0	
17	Annual	Million tonne-kilometre (TKM)	Total	Total transport	Slovakia													
18	Annual	Thousand tonnes	Total	Total transport	Austria	986.0	1006.0	905.0	741.0	903.0	933.0	778.0	937.0	834.0	889.0	680.0	640.0	
19	Annual	Thousand tonnes	Total	Total transport	Belgium	10714.0	10710.0	10122.0	8599.0	9071.0	9630.0	7202.0	8512.0	8247.0	8271.0	6374.0	6047.0	
20	Annual	Thousand tonnes	Total	Total transport	Bulgaria	190288.0	187404.0	180303.0	188158.0	192938.0	201129.0	151972.0	155695.0	156131.0	146160.0	146303.0	159990.0	
21	Annual	Thousand tonnes	Total	Total transport	Czechia	16378.0	16726.0	16622.0	17201.0	17467.0	16247.0	15462.0	18735.0	18824.0	18093.0	12548.0	13428.0	
22	Annual	Thousand tonnes	Total	Total transport	Germany	838.0	698.0	802.0	850.0	832.0	510.0	390.0	778.0	387.0	344.0	317.0	378.0	
23	Annual	Thousand tonnes	Total	Total transport	European Union - 27 countries (from 2020)	223170.0	226884.0	228489.0	221369.0	2221349.0	222731.0	107904.0	205066.0	188022.0	195093.0	183448.0	171689.0	
24	Annual	Thousand tonnes	Total	Total transport	Finland	53942.0	53950.0	54717.0	54575.0	54575.0	55125.0	503784.0	514014.0	495560.0	513925.0	488144.0	468460.0	
25	Annual	Thousand tonnes	Total	Total transport	France													
26	Annual	Thousand tonnes	Total	Total transport	Croatia	68542.0	66599.0	65139.0	62668.0	65389.0	62253.0	59529.0	64207.0	59879.0	57109.0	52001.0	47233.0	
27	Annual	Thousand tonnes	Total	Total transport	Hungary	9934.0	9623.0	8377.0	6642.0	6489.0	6221.0	5182.0	6491.0	7077.0	6548.0	4626.0	3765.0	
28	Annual	Thousand tonnes	Total	Total transport	Luxembourg	8135.0	7857.0	7825.0	8163.0	8224.0	8414.0	6925.0	8592.0	8803.0	8622.0	5921.0	5200.0	
29	Annual	Thousand tonnes	Total	Total transport	Netherlands	8596.0	8887.0	8390.0	7126.0	6325.0	6163.0	5741.0	6431.0	5755.0	6221.0	5652.0	4745.0	
30	Annual	Thousand tonnes	Total	Total transport	Poland	343029.0	348426.0	357788.0	349717.0	354587.0	359356.0	349480.0	347714.0	335892.0	357994.0	345479.0	327479.0	
31	Annual	Thousand tonnes	Total	Total transport	Romania	2574.0	3185.0	3699.0	3008.0	3911.0	3604.0	3119.0	2870.0	2517.0	2075.0	918.0	587.0	
32	Annual	Thousand tonnes	Total	Total transport	Sweden	27146.0	26958.0	27834.0	30020.0	30484.0	29043.0	29714.0	31051.0	30518.0	31120.0	28620.0	32102.0	
33	Annual	Thousand tonnes	Total	Total transport	Slovakia	8242.0	8107.0	7010.0	5721.0	4786.0	6996.0	5567.0	6430.0	6004.0	6013.0	5174.0	4670.0	
34																		
35																		
36																		
37																		

Fig. 1. Sample Table

Vocabulary construction begins with an analysis of the structure of the Eurostat tables. The leftmost columns and top rows contain dimension names, dimension values, measure names and/or units in which the measures are expressed. The top line of each table almost systematically contains temporal information, most commonly years, but also other time units such as months or quarters.

During the vocabulary construction process, category labels are assigned to attributes and values to indicate their membership in one of the four target categories: **Measures**, **Dimension Names**, **Dimension Values**, and **Unit Information**. The labels M , N , A , and U , respectively, are used throughout the following steps.

Temporal Information As illustrated in Figure 1, the temporal portion of the header is typically marked by the transition from a geopolitical dimension to a `\Time` or `\TIME.PERIOD` field. This structural pattern is exploited to identify the temporal portion of the top line and construct, for each table t , the set $D(t)$ of time intervals appearing in the table.

The implementation is provided in **Appendix A.1**.

String, Geographical and Value Information The attributes remaining after the split are used as **Dimension Names** and are tagged with N . For each such attribute, every non-numeric string value encountered is first added to $S(t)$, regardless of its content; $S(t)$ is therefore, at this stage, still duplicate-free but not yet restricted to non-geographical terms.

Each value is then checked against the NUTS 2024 vocabulary referenced in the assignment. If it matches an entry exactly, it is added to $Geo(t)$ and excluded from further processing, consistently with $Geo(t) \subset S(t)$. Otherwise, the value is classified as either **Unit Information** or **Dimension Value**, depending on whether it occurs under a **Unit of measure** or **Time frequency** column, and tagged with U or A respectively. $V(t)$ is thus obtained as $V(t) = S(t) \setminus Geo(t)$, with every element of $V(t)$ already carrying its U/A tag by construction, rather than through a separate filtering pass.

The implementation is provided in **Appendix A.2**.

Table Measures Let $\tau(t)$ denote the title of table t . Since the table title corresponds to its **measure**, it undergoes an analogous filtering process to the one applied to the strings extracted from the table: temporal and geographical mentions are identified and removed from $\tau(t)$, and the remaining text is added to $V(t)$ and tagged with "M".

The implementation is provided in **Appendix A.3**.

Global Vocabulary Once the vocabulary $V(t)$ has been constructed for each table, the individual vocabularies are merged into a single global vocabulary V , containing all distinct terms appearing in at least one table:

$$V = \bigcup_t V(t).$$

When the same term occurs in multiple tables, its associated information is combined so that the global vocabulary maintains all the tags assigned to that term across its occurrences.

The implementation is provided in **Appendix A.4**.

Vocabulary Classification So far, terms have been assigned one or more tags corresponding to the four target categories. During the merge into V , a term may therefore retain multiple tags if it was assigned to different categories in different tables.

Overlaps involving a "M" tag are left unchanged, since measures are primarily derived from table titles and are therefore given priority. For the remaining categories, the only overlap requiring an additional rule is between Dimension Values and Unit Information. Since the identification of Unit Information relies on a weak heuristic based on the attribute in which a value occurs, a term assigned both "A" and "U" is retained only with the "U" tag. This choice aims to reduce misclassification by assuming that such an overlap is more likely to result from a unit context not being correctly identified than from a genuine Dimension Value that should also be classified as Unit Information.

Once the overlaps have been resolved, the terms are grouped according to their final tags and the four resulting vocabularies are stored in four different files, as required by the assignment.

The implementation of this final step is provided in **Appendix A.5**.

2.2 Hierarchical Relationships Among Measures

It was observed that many measures within the same thematic domain share similar titles. This suggests using the textual structure of measure titles to organize them into hierarchical tree structures, where the shared prefix between two titles acts as a proxy for their conceptual closeness: measures sharing a longer portion of their titles tend to be closely related—potentially variants of one another—while those sharing only an initial, more generic portion tend to have little in common beyond belonging to the same broad topic.

However, lexical overlap alone can be misleading: the same words can occur in semantically unrelated contexts. To address this, the textual hierarchy is refined using sentence embeddings to identify and split off subtrees whose members share a title prefix but are not semantically coherent. This helps ensure that the resulting structure reflects conceptual similarity rather than surface-level wording. Under this construction, the relationship between two measures can be inferred from their position in the tree: measures sharing a more specific common path are considered more closely related, whereas measures that only share a distant common ancestor are considered related primarily at the level of the broader topic.

Further details on the implementation of this approach are provided in **Appendix A.6**.

2.3 Domain Clustering of Measures

Measures in the vocabulary are assigned to one of eight predefined thematic domains through a cluster-based classification approach, rather than by classifying each measure independently.

Assigning a domain to each individual measure via a separate LLM call would require one call per measure—close to seven thousand in this corpus—most of which would be redundant, since closely related measures (e.g., the same statistic broken down by different covariates) are likely to belong to the same domain. To reduce this cost, and to avoid assigning closely related measures to different domains, domain assignment is therefore performed once per *cluster* rather than once per measure. Clusters are obtained directly from the tree structure used to identify semantic relationships between measures, described in the previous section. Each tree therefore defines one cluster, and all measures belonging to that tree are treated as sharing a common domain.

For each cluster, a compact vocabulary is built by collecting the distinct non-stopword words appearing across its member measures, rather than using their full titles. This word list—rather than the individual measures themselves—is sent to the LLM together with a fixed set of candidate domains and a set of classification rules specified in the prompt. The model is instructed to return exactly one domain for each cluster. The resulting domain is then propagated to every individual measure in that cluster, yielding a final domain label per measure while requiring only as many LLM calls as there are clusters, rather than one call per measure.

Further details on the implementation of this approach are provided in **Appendix A.7**.

3 Evaluation

This section presents the results obtained for the domain clustering and evaluates the performance of the proposed approach. The analysis first reports the overall and per-domain classification metrics, and then examines the contribution of the clustering and LLM stages to the observed errors.

3.1 Vocabulary Classification

As described in 2.1, vocabulary classification is a deterministic, rule-based procedure driven entirely by the structural position of each term within a table (e.g., column position, column name, and presence in the NUTS dictionary). Therefore, an external manually labelled ground truth is not used for this step: such a ground truth would require prior knowledge of which terms should belong to each category, which is precisely the information that the proposed procedure aims to derive from the table structure.

Instead, the quality of the resulting vocabulary is assessed through a self-consistency test. For each category M , N , A , and U , a sample of terms is drawn and TF-IDF vectors, using unigrams and bigrams, are computed jointly over all sampled terms. For each term, its $k = 5$ nearest neighbours according to cosine similarity are then retrieved.

The self-consistency score of a term is defined as the fraction of its nearest neighbours that were assigned to the same category.

The intuition is that a coherent category should contain terms that are lexically similar to other terms assigned to the same category. However, self-consistency should not be interpreted as proof of a perfect separation between categories, but rather as evidence that the procedure produces categories that are internally coherent and reasonably well-separated from one another.

3.2 Domain Clustering

A ground truth for domain assignment was created by manually labeling the 2000 smallest tables provided in the sample, assigning each measure to one of eight domains. These domains were defined based on the official Eurostat classification¹ and correspond to the eight domains used in the LLM prompt, whose definitions are provided in **Appendix A.7**.

Given the labelled examples, standard classification metrics—accuracy, macro-averaged precision, recall, and F1-score—were used for the evaluation, together with a per-domain breakdown. These metrics were computed by comparing, for each ground-truth measure, its assigned domain with its predicted domain.

Since domain assignment is a two-stage process—measures are first grouped into trees by the similarity forest, and a single domain is then assigned to each tree—an aggregate accuracy score alone cannot distinguish between two different sources of error. A wrong final domain may result either from the clustering step

¹ See the official Eurostat database

merging measures from genuinely different domains into the same tree (*upstream error*), or from the LLM assigning an incorrect domain to an otherwise coherent cluster (*downstream error*).

To investigate these two sources of error, the *purity* of each cluster is computed as the fraction of its ground-truth measures belonging to the cluster’s majority domain. Low cluster purity indicates that the clustering step has grouped measures from different domains together. The incorrect measures are then attributed to either the clustering step or the LLM’s domain choice, allowing the relative contribution of the two error sources to be quantified. This analysis therefore assesses the extent to which the final accuracy is limited by the similarity forest’s ability to group measures from the same domain versus the LLM’s ability to assign the correct domain once a coherent cluster is provided.

3.3 Semantic Relationships

Building a ground truth over individual pairs of measures, as would be required to directly evaluate the relationships identified in **Section 2.2**, is impractical at this corpus size. Consequently, rather than attempting a full pairwise ground truth, semantic relationships are evaluated indirectly by reusing the same per-measure domain labels built for the previous step as an (imperfect) proxy for relatedness. Two measures placed in the same tree are expected, at a minimum, to belong to the same domain; therefore, domain agreement provides evidence that the measures grouped within a tree are related at least at the level of their broader thematic domain.

Two complementary checks are performed on this basis.

The first evaluates the purity of the final trees. For each tree, purity measures whether its members share the same ground-truth domain. Both the fraction of fully pure trees and the mean purity, weighted by the number of measures per tree, are reported. A tree containing measures from multiple domains indicates that the similarity-based splitting did not fully separate measures that should not have remained grouped together.

The second evaluation targets the splitting step specifically, rather than the pipeline as a whole. Two measures that share the same domain and initially belong to the same, *original* single-word root tree should not be split into different subtrees. Conversely, two measures belonging to different domains should ideally be separated.

For every such pair, the outcome is classified as *preserved*, *lost*, *correctly separated*, or *still incorrectly merged*. From these outcomes, a relationship recall, defined as the proportion of preserved same-domain relationships, and a separation rate, defined as the proportion of correctly separated different-domain pairs, are derived.

4 Results

4.1 Vocabulary Classification

The self-consistency evaluation was performed by randomly sampling 5,000 terms from each category.

Category	Mean self-consistency
<i>A</i> (Dimension Values)	0.676
<i>M</i> (Measures)	0.893
<i>N</i> (Dimension Names)	0.275
<i>U</i> (Unit Information)	0.790
Overall	0.755

Category	Neighbour category			
	<i>A</i>	<i>M</i>	<i>N</i>	<i>U</i>
<i>A</i>	0.676	0.245	0.023	0.055
<i>M</i>	0.030	0.893	0.055	0.022
<i>N</i>	0.071	0.596	0.275	0.058
<i>U</i>	0.043	0.139	0.028	0.790

Category *M* is the most self-consistent of the four, with a score of 0.893. This is consistent with the nature of measure terms, which are typically long, multi-word phrases describing specific statistical concepts. As a result, their TF-IDF representations, based on unigrams and bigrams, tend to contain distinctive combinations of words. Category *U* also shows a relatively high self-consistency, with a score of 0.790. Its main source of overlap is with *M*: 13.9% of the nearest neighbours of *U* belong to *M*, compared with only 4.3% belonging to *A*. This may be explained by the fact that some unit descriptions, particularly percentage- or index-based expressions, share vocabulary with measure titles.

Category *A* achieves a self-consistency score of 0.676. The neighbour distribution shows that 24.5% of its nearest neighbours belong to *M*, making this its main source of lexical overlap. This is not unexpected, since some dimension values are descriptive, multi-word labels referring to sectors, activities, or indicator-related concepts. Such terms may share vocabulary with measure titles describing the corresponding statistical phenomena.

Category *N* has the lowest self-consistency, with a score of 0.275. The neighbour distribution provides an explanation for this result: 59.6% of the nearest neighbours of terms classified as *N* belong to *M*, whereas only 7.1% belong to *A*. This suggests that dimension names in the corpus are often short and generic

expressions that recur, either verbatim or approximately, within longer measure titles. For example, a dimension name such as *"Health care"* may appear as part of a measure title such as *"Health care expenditure by financing scheme"*. Consequently, N terms may be lexically closer to M than to the dimension values associated with them.

The observed overlap should not necessarily be interpreted as evidence of incorrect classification. The four categories are generated from structurally distinct sources in the pipeline: attribute names, dimension values, processed table titles, and unit information. Therefore, the neighbour distribution primarily reflects lexical relationships between categories rather than overlap in their construction. Overall, the results indicate that M , A , and U are reasonably coherent, while the lower self-consistency of N is mainly associated with its strong lexical similarity to measure titles.

4.2 Domain Clustering

As outlined in **Appendix A.6**, the construction of the similarity forest depends on two main parameters that jointly determine the number and granularity of the resulting clusters, which in turn affect both the number of LLM calls required and, indirectly, the accuracy of domain assignment:

```
params = {"max_split_depth": 2, "cos_sim_threshold": 0.68}
```

This configuration was selected after several trials, balancing the number of LLM calls, kept as low as possible, against classification accuracy.

A higher cosine similarity threshold and a greater maximum split depth both lead to more, finer-grained clusters—and therefore more LLM calls—but also to more homogeneous clusters and potentially higher classification accuracy.

Lowering either parameter has the opposite effect: a lower threshold tolerates more semantic divergence within a cluster before splitting it, while a shallower maximum depth stops the outlier-detection check earlier, leaving deeper branches merged regardless of how semantically divergent they may be.

However, cluster size also affects the quality of the LLM’s domain assignment. When a cluster contains a large number of measures, its vocabulary may include terms that are individually associated with different domains or are less representative of the underlying topic. These terms can introduce ambiguity and potentially lead the LLM towards an incorrect domain. Conversely, a more homogeneous and focused cluster provides a more informative vocabulary, allowing terms that are more characteristic of the actual domain to provide a stronger signal and reduce the influence of less relevant terms.

The chosen combination was therefore found to offer the best trade-off between the number of LLM calls, cluster homogeneity, and classification accuracy.

Table 1 reports the aggregate classification metrics obtained by comparing, for each of the 1833 ground-truth measures, the predicted domain against the manually assigned one.

Table 1. Overall domain classification metrics.

Metric	Value
Total measures compared	1833
Correct classifications	1618
Incorrect classifications	215
Accuracy	88.27%
Macro Precision	88.99%
Macro Recall	80.76%
Macro F1-score	83.80%

All aggregate metrics are above 80%, with precision being particularly high. A per-domain breakdown, reported in Table 2, shows that *Tech & Innovation* and *Living Well* have the weakest overall performance, particularly in recall. This is mainly attributable to the LLM’s difficulty in recognizing these domains from ambiguous table titles, a problem that is especially pronounced for *Tech & Innovation*.

Table 2. Per-domain classification metrics.

Domain	Precision	Recall	F1-score	Support
Money & Markets	0.831	0.888	0.859	393
People, Work & Living Conditions	0.842	0.940	0.888	532
Health & Safety	0.915	0.956	0.935	181
Learning & Education	0.956	0.763	0.849	114
Environment & Natural Resources and Production	0.996	0.888	0.939	260
Transports	0.957	0.966	0.961	206
Tech & Innovation	0.766	0.590	0.667	83
Living Well	0.857	0.469	0.606	64

As described in Section 3.2, the two-stage nature of the domain assignment process allows the incorrect classifications to be attributed to either the clustering step (upstream error) or the LLM’s domain choice (downstream error). Table 3 reports this breakdown, together with the purity of the clusters covered by the ground truth.

The high mean cluster purity (97.27%, with 854 out of 878 clusters fully pure) indicates that the similarity forest is largely successful at grouping measures from the same domain together upstream. Consistently, the large majority of the remaining errors (80%) are downstream, i.e. due to the LLM assigning an incorrect domain to an otherwise coherent cluster, rather than to the clustering step merging measures from different domains. This suggests that the “same cluster $\Rightarrow$ same domain” assumption underlying the pipeline is effective in the

Table 3. Cluster purity and error attribution.

Metric	Value
Clusters covered by ground truth	878
Pure clusters (100% single domain)	854 / 878
Mean cluster purity (weighted by # measures)	97.27%
Incorrect measures (total)	215
– downstream (LLM domain choice)	172 (80.0%)
– upstream (clustering)	43 (20.0%)

large majority of cases, and that the main bottleneck for overall accuracy lies in the LLM’s domain classification rather than in the similarity-based clustering.

4.3 Semantic Relationships

For the last step, the same two parameters control the construction of the similarity forest:

```
params = {"max_split_depth": 2, "cos_sim_threshold": 0.57}
```

Unlike in step 7, where the evaluation focuses solely on the domain eventually assigned to each cluster, the evaluation here is concerned directly with the pairwise relationships encoded by the tree structure itself, and the trade-off the two parameters control shifts accordingly.

Larger, coarser clusters increase the risk of merging measures that should not be related (impurity), while smaller, more aggressively split clusters increase the risk of separating measures that should remain related (lost relationships).

The two failure modes pull the parameters in opposite directions, and no configuration found during tuning avoids both simultaneously: the results reported below represent the best trade-off identified, with both checks converging to roughly 70–75%, which we regard as a genuine limitation of the method rather than a configuration that can still be improved by further tuning alone.

Metric	Value
Ground-truth measures matched to a tree	1 833 / 1 833 (100.0%)
Fully pure trees (multi-measure trees only)	224 / 259 (86.5%)
Mean purity, weighted by # measures	0.929

Of the 259 final trees containing more than one ground-truth measure, 224 (86.5%) are fully pure, i.e. every ground-truth measure they contain shares the same domain; the purity averaged across all multi-measure trees, weighted by the number of measures each contains, is 0.929. The 13.5% of trees that are not fully pure indicate cases in which the similarity threshold was not strict enough to separate two branches that, in the ground truth, actually belong to different domains.

Pair type	Total	Outcome	Rate
Same-domain (true relationship)	8 239	Preserved	6 180 (75.0%)
		Lost	2 059 (25.0%)
Different-domain (spurious relationship)	4 942	Correctly separated	3 600 (72.8%)
		Still merged	1 342 (27.2%)

Restricting the comparison to pairs of ground-truth measures that started out in the same original, single-word root tree, 8 239 such pairs share the same domain and therefore encode a true relationship; of these, 6 180 (75.0%) are preserved in the same final tree after splitting, while 2 059 (25.0%) are lost – separated into different trees despite belonging to the same domain. Symmetrically, of the 4 942 pairs that started in the same root tree but belong to different domains, 3 600 (72.8%) are correctly separated by splitting, while 1 342 (27.2%) remain incorrectly merged in the same final tree.

Taken together, these two checks show that splitting is only moderately effective at distinguishing genuine same-domain relationships from spurious same-root-word coincidences: at the chosen configuration, it removes roughly three quarters of the spurious relationships it encounters, but at the cost of also discarding about a quarter of the genuine ones – a symmetric error rate that persisted across the configurations explored during tuning, rather than one that could be resolved by shifting the threshold in either direction alone.

5 Conclusion

This work presented a pipeline for extracting, structuring, and organizing the vocabulary of a large corpus of Eurostat statistical tables. The pipeline addressed three main tasks: constructing a global vocabulary partitioned into Measures, Dimension Names, Dimension Values, and Unit Information; organizing measures into a similarity forest and assigning them to eight thematic domains; and identifying semantic relationships between measures. The three outputs were evaluated using intrinsic consistency measures and manually labelled samples, where available, despite the limited availability of an external ground truth.

The results support the effectiveness of the proposed approach while highlighting its limitations. Vocabulary classification produced generally coherent categories, with the lower consistency of Dimension Names explained by their lexical overlap with measure titles and dimension values. Domain clustering achieved an accuracy of 88.27 and a macro F1-score of 83.80, indicating that most residual errors originated in the downstream LLM classification rather than in the similarity forest. Semantic relationship identification achieved recall and separation rates of approximately 70–75% on prefix-based grouping and embedding similarity.

A central design choice was to reuse the same similarity forest for both domain clustering and semantic relationship identification. This made the pipeline more computationally tractable and ensured that both tasks were based on a shared representation of measure similarity. Moreover, restricting the LLM to a fixed set of eight predefined domains helped prevent the generation of unsupported or hallucinated categories.

Several improvements remain possible. In particular, making the forest less sensitive to word order and morphological variation could improve the identification of semantic relationships. For domain clustering, refining the LLM prompt and providing additional cluster-level context appears more promising than further tuning the forest, given that most classification errors were found to occur downstream. Overall, the proposed pipeline provides a scalable and coherent framework for building and enriching statistical vocabularies from large collections of structured data.

A Steps implementation

A.1 Temporal Information

The following function implements the identification of the temporal portion of the table header. The function searches for the time marker and splits the attributes into the portion preceding it (`attributes[:candidate]`) and the temporal portion following it (`attributes[candidate:]`).

If no such marker is found, the table is conservatively treated as having no temporal columns, and $D(t) = \emptyset$.

```

1
2 def split_attributes(attributes):
3
4     candidate = None
5
6     for i, attribute in enumerate(attributes):
7         if has_time_marker(attribute):
8             candidate = i + 1
9             break
10
11     if candidate is not None:
12         return set(attributes[:candidate]), set(attributes[candidate:])
13
14     return set(attributes), set()
15

```

Source: *utils.py*

A.2 String, Geographical and Value Information

The following code picks up from the splitting function described in Appendix A.1 and constructs the $S(t)$ set of textual strings appearing in the leftmost columns of each table.

Attribute names are stripped of any `\Time/\TIME_PERIOD` marker before being added to $S(t)$ and tagged, as it is merely a Eurostat boundary artifact. The corresponding column values are then filtered to retain only strings and duplicate values are removed using `unique()`. This avoids repeatedly processing identical values that frequently occur multiple times within the same Eurostat column, while preserving all distinct values relevant to the vocabulary.

```

1
2 ...
3
4 for attribute in string_attributes:
5
6     cleaned_attribute = strip_time_marker(attribute)
7

```

```

8      # 2.1 Attribute name -> N
9      S_t[file.name].add(cleaned_attribute)
10
11     if cleaned_attribute not in V_t[file.name]:
12         V_t[file.name][cleaned_attribute] = set()
13     V_t[file.name][cleaned_attribute].add("N")
14
15     values = tables[attribute].dropna()
16     values = values[values.apply(lambda v: isinstance(v,
17         ↪ str))].unique()
18
19     for value in values:
20
21         if not isinstance(value, str) or is_numeric(value):
22             continue
23
24         # 2.2 String value
25         S_t[file.name].add(value)
26
27         # 3.1 Geographic value
28         ...

```

Source: *pipeline.py*

The geographic check described in section 2.1 is implemented as follows: matching values are moved to $Geo(t)$ and excluded from further processing.

```

1  # ... continues from the block above, inside the same loop over `values`
2
3      # 3.1 Geographic value
4      if value in dict_nuts:
5          Geo_t[file.name].add(value)
6          continue
7
8      # 3.2 Non-geographic value ->  $V(t) = S(t) \setminus Geo(t)$ 
9      if value not in V_t[file.name]:
10         V_t[file.name][value] = set()
11
12     ...

```

Source: *pipeline.py*

The U/A distinction described in section 2.1 is implemented below.

```

1  # ... continues from the block above, still inside the same loop
2
3      if attribute in {"Unit of measure", "Time frequency"}:
4          V_t[file.name][value].add("U")
5      else:

```

```

6         V_t[file.name][value].add("A")
7
8         ...

```

Source: pipeline.py

A.3 Table titles

The following function processes each table title by separating geographical and temporal information from the remaining text. The geographical vocabulary is provided as an input, while temporal information is identified using a regular expression for four-digit years between 1900 and 2100.

```

1
2 def parse_title(title, geo_vocab, max_geo_words=6):
3
4     words = title.split()
5     n = len(words)
6
7     geo = set()
8     temporal = set()
9     remaining_words = []
10
11     i = 0
12     while i < n:
13
14         matched_span = None
15
16         # Try progressively shorter phrases starting at i, longest
17         ↪ first,
18         # so multi-word geo entities are preferred over any partial
19         ↪ match.
20         max_span = min(max_geo_words, n - i)
21         for span in range(max_span, 0, -1):
22             phrase_words = words[i:i + span]
23             # only strip trailing punctuation from the phrase's last
24             ↪ word
25             phrase = " ".join(phrase_words[:-1] +
26                             [phrase_words[-1].strip(".,;:()[]{}")])
27
28             if phrase in geo_vocab:
29                 geo.add(phrase)
30                 matched_span = span
31                 break
32
33     if matched_span is not None:
34         i += matched_span
35         continue

```

```

33     word = words[i]
34     clean_word = word.strip(".,;:() []{}")
35
36     # Year
37     years = re.findall(r'\b\d{4}\b', clean_word)
38
39     if any(1900 <= int(year) <= 2100 for year in years):
40         temporal.update(years)
41         i += 1
42         continue
43
44     remaining_words.append(word)
45     i += 1
46
47     remainder = " ".join(remaining_words)
48
49     return remainder

```

Source: `utils.py`

The title is first split into individual words and then processed from left to right. At each position, the function first attempts to match the longest possible sequence of up to six words against the geographical vocabulary. This allows multi-word geographical entities to be recognized as a single entity and prevents shorter partial matches from being selected. If a geographical entity is found, its words are skipped. Otherwise, the current word is checked for a four-digit year between 1900 and 2100, corresponding to the range of years that can occur in the tables. Such years are stored as temporal information and removed from the title. All other words are retained in ‘remaining_words’. Finally, the remaining words are joined to form ‘remainder’, which contains the portion of the title that is neither geographical nor temporal information. This remainder is subsequently added to $V(t)$ and tagged with "M".

```

1
2 ...
3
4 # 4. Remaining title
5     if title_remaining:
6
7         # Store the measure with its M tag
8         if title_remaining not in V_t[file.name]:
9             V_t[file.name][title_remaining] = set()
10            V_t[file.name][title_remaining].add("M")
11
12 ...
13

```

Source: `pipeline.py`

A.4 Global Vocabulary

The construction of the global vocabulary V described in Section 2.1 is implemented below.

```

1  ...
2  ...
3
4  # 5-6. Global mapped vocabulary
5      V = {}
6
7      for vocabulary in V_t.values():
8          for term, tags in vocabulary.items():
9              V.setdefault(term, set()).update(tags)
10
11  ...
12

```

Source: pipeline.py

A.5 Vocabulary Classification

The following code handles the overlap between Dimension Values and Unit Information identified during the construction of the global vocabulary. If a term has both "A" and "U" tags, only the "U" tag is retained, as described in Section 2.1.

```

1  ...
2  ...
3
4  for term, tags in V.items():
5      if tags == {"U", "A"}:
6          V[term] = {"U"}
7
8  (OUTPUT / "vocabulary_by_tag").mkdir(parents=True, exist_ok=True)
9  save_vocabulary_by_tag(V, OUTPUT / "vocabulary_by_tag")
10
11  ...
12

```

Source: pipeline.py

```

1
2  def save_vocabulary_by_tag(V, output_dir):
3
4      tag_files = {
5          "M": "measures.csv",
6          "N": "dimension_names.csv",
7          "A": "dimension_values.csv",
8          "U": "units.csv"

```

```

9     }
10
11     for tag, filename in tag_files.items():
12
13         terms = []
14
15         for term, tags in V.items():
16             if tag in tags:
17                 terms.append(term)
18
19         df = pd.DataFrame({"term": sorted(terms)})
20
21         df.to_csv(output_dir / filename, index=False)

```

Source: `utils.py`

A.6 Similarity forest Construction

The first step consists in constructing a raw tree from the tokenized measure titles. Initially, measures are grouped according to the first word of their titles, which becomes the root of the corresponding tree. Each node stores its children in a dictionary, where each key corresponds to a word, and keeps track of measures whose titles end exactly at that node. For each measure, the remaining tokens, starting from the position following the root, are then traversed sequentially. If a token is not already present among the children of the current node, a new node is created. The measure index is then stored in the terminal node.

```

1
2 class RawNode:
3     def __init__(self):
4         self.children = {}          # token -> RawNode
5         self.terminal_measures = [] # indices of measures whose title
6                                     ↪ ends exactly here
7
8 def build_raw_tree(indices, tokens, start_pos):
9     root = RawNode()
10    for idx in indices:
11        node = root
12        for tok in tokens[idx][start_pos:]:
13            if tok not in node.children:
14                node.children[tok] = RawNode()
15            node = node.children[tok]
16        node.terminal_measures.append(idx)
17    return root

```

Source: `clustering.similarity_forest.build_forest.py`

This produces a prefix tree in which measures sharing the same sequence of words follow the same path until their titles diverge. The resulting structure therefore directly represents the textual relationships between the measure titles.

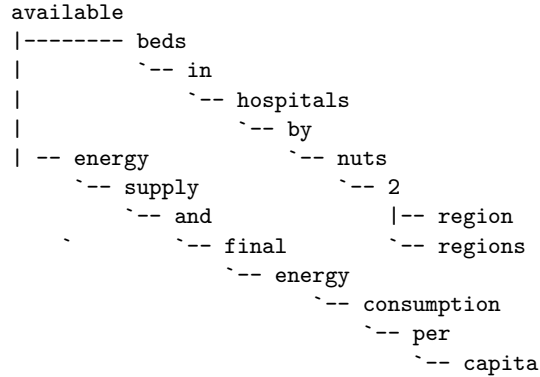


Fig. 2. Example of a raw tree constructed from measure titles.

The raw tree can contain long sequences of nodes having only one child, which do not represent actual branching points. These chains are therefore compressed into a single edge labelled with the corresponding sequence of words. The `compress` function traverses the raw tree and progressively merges consecutive single-child nodes until a branching point or a terminal node is reached.

```

1
2 class CompressedNode:
3     def __init__(self, edge_label="", path=()):
4         self.edge_label = edge_label # the (possibly multi-word) phrase
5         ↪ on the edge INTO this node
6         self.path = path # full sequence of words from the
7         ↪ tree's true root to this node
8         self.children = []
9         self.terminal_measures = [] # measures whose title ends
10        ↪ exactly at this node
11
12 def compress(raw_node, path_prefix):
13     node = CompressedNode(path=path_prefix)
14     node.terminal_measures = list(raw_node.terminal_measures)
15
16     for tok, child_raw in raw_node.children.items():
17         chain_tokens = [tok]
18         cur = child_raw
19         while len(cur.children) == 1 and not cur.terminal_measures:
20             (next_tok, next_node), = cur.children.items()

```

```

19         chain_tokens.append(next_tok)
20         cur = next_node
21
22         edge_label = " ".join(chain_tokens)
23         child_path = path_prefix + tuple(chain_tokens)
24         child_compressed = compress(cur, child_path)
25         child_compressed.edge_label = edge_label
26         node.children.append(child_compressed)
27
28     return node

```

Source: clustering.similarity_forest.build_forest.py

Each compressed node stores both the label of the edge leading to it and the complete sequence of words from the root to that node. This produces a more compact representation in which the nodes mainly correspond to points where measures share a prefix and subsequently diverge.

```

available
|-- beds in hospitals by nuts 2
|   |-- region
|       `-- regions
`-- energy supply and final energy consumption per capita

```

Fig. 3. Example of a compressed tree constructed from measure titles.

Since the initial grouping is based on the textual prefixes of the titles, semantic information is introduced to identify branches that are too weakly related to remain in the same tree. Sentence embeddings are computed once for all measure titles using the `all-MiniLM-L6-v2` model and normalized so that cosine similarity can be used to compare them.

At each branching point, the measures contained in each child subtree are collected. The centroid of the embeddings belonging to a branch is then compared with the centroid obtained from all other sibling branches. If the resulting cosine similarity is lower than or equal to the predefined threshold, the branch is considered an outlier and is separated from the current tree. This process is applied recursively to the resulting subtrees.

The default parameters define a cosine similarity threshold of 0.50 and do not impose a maximum splitting depth. The parameter `max_examples` only controls the number of example measures stored in the output and does not affect the splitting procedure.

```

1
2 def embed_terms(terms, model_name="all-MiniLM-L6-v2"):
3     model = SentenceTransformer(model_name)

```

```

4     embeddings = model.encode(terms, normalize_embeddings=True,
5     ↪ show_progress_bar=True)
6     return np.asarray(embeddings)
7
8     def centroid(embeddings, indices):
9         vecs = embeddings[indices]
10        c = vecs.mean(axis=0)
11        norm = np.linalg.norm(c)
12        return c / norm if norm > 0 else c
13
14
15    def cosine_sim(a, b):
16        return float(np.dot(a, b))
17
18
19
20
21    def find_outlier_children(node, embeddings, params):
22        if len(node.children) < 2:
23            return []
24
25        if params["max_split_depth"] is not None and len(node.path) >
26        ↪ params["max_split_depth"]:
27            return []
28
29        child_indices = [collect_subtree_measures(c) for c in node.children]
30
31        outliers = []
32        for i, indices_i in enumerate(child_indices):
33            rest_indices = []
34            for j, indices_j in enumerate(child_indices):
35                if j != i:
36                    rest_indices.extend(indices_j)
37
38            if not rest_indices:
39                continue
40
41            centroid_i = centroid(embeddings, indices_i)
42            centroid_rest = centroid(embeddings, rest_indices)
43            sim = cosine_sim(centroid_i, centroid_rest)
44
45            if sim <= params["cos_sim_threshold"]:
46                outliers.append(node.children[i])
47
48        return outliers

```

Source: clustering.similarity_forest.build_forest.py

Once the outlier children of a node have been identified, they are detached from the current tree and recursively re-processed as independent trees, while the remaining children continue to be processed within the same tree. This procedure is applied at every branching point of every tree, so that a single split can, in turn, expose further splits deeper in the resulting subtrees.

```

Tree 1
available
  |-- beds in hospitals by nuts 2
    |-- region
    |-- regions

Tree 2
available
  |-- energy supply and final energy consumption per capita

```

Fig. 4. Trees obtained after semantic splitting. The semantically unrelated energy branch is separated from the hospital-bed measures.

A.7 Domain Clustering of Measures

The domain classification starts from the similarity forest constructed in the previous step. Since each tree in the forest represents a cluster of related measures, the first step is to recover the association between each individual measure and the tree to which it belongs.

First, the `load_forest` function reads the CSV representation of the similarity forest and constructs two mappings. The first, `measure_to_tree`, associates each measure with the `tree_id` of the tree in which it terminates. This is identified through the `full_path` column and the `n_terminal_here` value, which indicates whether at least one measure ends at that node.

The second mapping, `tree_root_prefix`, associates each tree with the label of its root edge and is used only as a human-readable identifier in the output.

```

1
2 def load_forest(forest_path):
3
4     forest = pd.read_csv(forest_path)
5
6     measure_to_tree = {}
7     tree_root_prefix = {}
8
9     for _, row in forest.iterrows():
10
11         if row["depth"] == 0:

```

```

12         tree_root_prefix[row["tree_id"]] = row["edge_label"]
13
14     if row["n_terminal_here"] >= 1:
15         key = tuple(str(row["full_path"]).split(" > "))
16         measure_to_tree[key] = row["tree_id"]
17
18     return measure_to_tree, tree_root_prefix

```

Source: clustering.clustering.py

Second, the `build_cluster_words` function constructs a compact vocabulary for each cluster. Measures are first grouped by their `cluster` identifier. For every measure, alphabetic words are extracted using a regular expression and converted to lowercase. English stopwords are discarded using the `ENGLISH_STOP_WORDS` list from `scikit-learn`. Duplicate words within the same cluster are also removed while preserving their first occurrence.

```

1  def build_cluster_words(cluster_prefixes, llm_files_path):
2
3
4      cluster_rows = []
5
6      for cluster_id, group in cluster_prefixes.groupby("cluster",
7      ↪ sort=True):
8
9          words = []
10         seen_words = set()
11
12         for measure in group["measure"].dropna():
13
14             measure_words = re.findall(r"[A-Za-z]+", measure)
15
16             for word in measure_words:
17                 word_normalized = word.lower()
18                 if word_normalized in ENGLISH_STOP_WORDS:
19                     continue
20                 if word_normalized not in seen_words:
21                     seen_words.add(word_normalized)
22                     words.append(word)
23
24             cluster_rows.append({"cluster": cluster_id, "words": words})
25
26         cluster_words = pd.DataFrame(cluster_rows, columns=["cluster",
27         ↪ "words"])
28         cluster_words["cluster"] = cluster_words["cluster"].astype(int)
29         cluster_words =
30         ↪ cluster_words.sort_values("cluster").reset_index(drop=True)
31
32     cluster_words.to_csv(llm_files_path / "cluster_words.csv",
33     ↪ index=False)

```

```

30
31     return cluster_rows

```

Source: `clustering.clustering.py`

The resulting list therefore contains the distinct non-stopword words appearing in the measures belonging to each tree. These cluster-level word lists are stored in `cluster_words.csv` and are subsequently used as input to the LLM instead of sending the complete measure titles.

For example, consider the following measures belonging to cluster 20:

```

20,Agricultural holdings,Agricultural holdings by age of holder
20,Agricultural holdings,Agricultural holdings by age of manager
20,Agricultural holdings,Agricultural holdings by agricultural area
20,Agricultural holdings,Agricultural holdings by crops
20,Agricultural holdings,Agricultural holdings with livestock

```

The `build_cluster_words` function extracts the words appearing in these measures, removes English stopwords, and eliminates duplicates within the cluster. The resulting compact vocabulary is therefore:

```

20,"['Agricultural', 'holdings', 'age', 'holder', 'manager',
'area', 'crops', 'livestock']"

```

This compact representation captures the main terms characterizing the cluster while avoiding sending the complete set of measure titles to the LLM.

Finally, once the cluster-level vocabularies have been constructed, the `prompt` function sends them to the LLM together with the candidate domains and the classification rules defined in the prompt. The model returns one domain assignment for each cluster. The following code defines the domain descriptions and constructs the messages sent to the LLM:

```

1
2 DOMAINS = {
3     "Money & Markets": "Financial markets and instruments, national
    ↳ accounts, public and private debt, exchange/interest rates,
    ↳ international trade, business demography and enterprise/government
    ↳ activity.",
4     "People, Work & Living Conditions": "Population structure, households,
    ↳ births, deaths and migration, housing conditions, employment, wages,
    ↳ working conditions, poverty, income inequality, social protection.",
5     "Health & Safety": "Physical and mental health, healthcare, mortality
    ↳ and survival as health outcomes, workplace safety, crime,
    ↳ gender-based violence.",
6     "Learning & Education": "Education at all levels, lifelong learning,
    ↳ skills acquired.",
7     "Environment & Natural Resources and Production": "Environment, energy,
    ↳ land use, agriculture, fisheries, forestry, natural resource
    ↳ sustainability.",

```

```

8 "Transports": "Transport of people and goods across all modes (air,
  ↳ maritime, rail, road, inland waterways).",
9 "Tech & Innovation": "Digital technologies and ICT, including internet
  ↳ access, digital skills, ICT specialists, R&D, innovation, high-tech
  ↳ activities, patents, trademarks, designs, and intellectual
  ↳ property.",
10 "Living Well": "Culture and cultural activities participation, sport,
  ↳ tourism, leisure, recreation, time use, well-being, and secondary
  ↳ activities."
11 }
12
13 def build_message(terms):
14     return [
15         {
16             "role": "system",
17             "content": """"You are a system that associates each term with EXACTLY
  ↳ ONE domain from the list below.
18
19             RULES:
20             - For EVERY list of terms you receive, choose EXACTLY ONE
  ↳ domain. Never assign more than one domain to the same list,
  ↳ even if it seems to fit several.
21             - Assign each statistic to the most appropriate domain based on
  ↳ what is being measured, not on its disaggregation variables.
22             - Classify according to the substantive phenomenon being
  ↳ measured, not according to the generic context in which it
  ↳ appears.
23             - I'll give you a dictionary with the domains as keys and
  ↳ explanations of what each represents.
24             - Respond ONLY with single chosen domain name. No explanation,
  ↳ no markdown, no extra text.
25
26             """,
27         },
28         {
29             "role": "user",
30             "content": f""""
31             Domains: {DOMAINS}
32             list_of_terms: {terms}
33             """,
34         }
35     ]

```

Source: *clustering.llm_requests.py*

If the response does not match one of the allowed domains exactly, an exception is raised and the processing is interrupted for that cluster.

The resulting cluster-level domain assignments are then merged with the measure-to-cluster mapping using the common `cluster` identifier. This propagates the domain assigned to each tree to all measures belonging to that tree,

producing a direct measure-to-domain association. The final result is stored in `measure_domain.csv`.